Book Recommendation System

Final Project Report
ECS 170 SQ 2024

Christina Phan[1], Jack Schonherr[1], Lauren Mandell[1], Madeleine Clark[1], Sajid Mohammad[1], Vedis Wu[1], and Vibha Chandrasekar[1]

[1]Department of Computer Science, University of California, Davis

June 12, 2024

1 Introduction

Our final project is a book recommendation system, developed as a web application that allows users to input a variable number of books they like and receive personalized book recommendations based on those inputs. The significance of this project stems from the widespread use of recommendation systems in various industries to provide tailored content suggestions to users. This technology is employed by major platforms like Netflix, Spotify, and Amazon to enhance user experiences by offering personalized content.

This project has significantly deepened our understanding of recommendation systems and their practical applications. Unlike most existing book recommendation systems that rely on user-based inputs, our approach focuses on book-based inputs. This unique approach aligns more closely with how readers often choose books based on their affinity for certain titles, authors, or genres. Relying on book-based eliminates the need to collect and store user data, thereby addressing user privacy concerns.

With the vast number of books available today, finding the right book to read can be overwhelming for readers. Our book recommendation system aims to simplify this process by helping users discover new books that match their preferences, thus making their reading experience more enjoyable and efficient. Additionally, this system benefits authors by connecting their works with readers who are likely to be interested in their books. In essence, our project aspires to serve as a reliable guide for avid readers seeking their next great read, leveraging advanced recommendation techniques to deliver high-quality, personalized book suggestions.

2 Background

2.1 Concepts

To fully understand the development and functionality of our book recommendation system implementations, readers should be familiar with several key concepts related to recommendation systems.

2.2 Recommender Systems and Their Challenges

Recommender systems are algorithms designed to suggest items to users based on various types of data[1]. These systems are widely used in various industries to enhance user experiences by providing personalized content recommendations. Examples include suggesting movies on Netflix, products on Amazon, or songs on Spotify. The primary goal of recommender systems is to predict the preferences of users and recommend items they are likely to enjoy.

[1]Paul Resnick and Hal R. Varian. 1997. Recommender systems. Commun. ACM 40, 3 (1997), 56–58. DOI: https://doi.org/10.1145/245108.245121

One challenge collaborative recommendation systems face is the cold start problem. The cold start problem occurs when there is insufficient data about new users or new items[2]. New users and items lack ratings; This lack of data makes it difficult for the system to make accurate recommendations. Addressing this problem often involves using hybrid approaches or incorporating additional data sources to improve initial recommendations.

Another challenge is balancing the diversity and relevance of recommendations[3]. While it is important to suggest items that are highly relevant to the user's preferences, it is also beneficial to introduce a degree of diversity to help users discover new interests. An overly narrow recommendation list can lead to a monotonous user experience, whereas too much diversity can result in irrelevant suggestions.

2.3 Collaborative vs. Content-Based Filtering, and Hybrid Approaches

Collaborative filtering makes recommendations based on the preferences of similar users[4]. Content-based filtering recommends items by comparing the content of such items and the user's preferences[4]. Hybrid recommendation systems combine collaborative and content-based filtering techniques to leverage the strengths of both methods[4].

2.4 Neural Networks

Neural networks are a class of machine-learning models inspired by the human brain. They consist of interconnected layers of nodes (neurons) that process input data to produce output predictions. In recommendation systems, neural networks can be used for tasks like collaborative filtering, where they learn to predict user preferences based on past interactions. Specifically, Neural Collaborative Filtering (NCF) models user neural networks to capture the complex relationships between users and items, providing more accurate and personalized recommendations[5].

2.5 Matrix Factorization

Matrix factorization is a machine-learning technique that aims to predict missing values in a sparse matrix through matrix decomposition. It creates two smaller matrices that, when multiplied together, closely approximate the original values in the sparse matrix while filling in the empty entries based on learned features.

3 Methodology

We conducted exploratory data analysis on our dataset and implemented three recommendation models: a hybrid model combining collaborative and content-based filtering, a Neural Collaborative Filtering (NCF) model, and a Matrix Factorization model.

3.1 Dataset

The dataset of this project was derived from the goodbooks-10k-extended dataset[6], which includes information about roughly ten thousand books and over six million user ratings from Goodreads. For each book, the dataset provides information like the title, authors, description, genre, page count, publication date, average

[2]Cold Start Problem: Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and Metrics for Cold-start Recommendations. Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval.

[3]Diversity and Relevance of Recommendations: McNee, S. M., Riedl, J., & Konstan, J. A. (2006). Being Accurate is Not Enough: How Accuracy Metrics Have Hurt Recommender Systems. CHI '06 Extended Abstracts on Human Factors in Computing Systems.

[4]Burke, R. Hybrid Recommender Systems: Survey and Experiments. User Model User-Adap Inter 12, 331–370 (2002). https://doi.org/10.1023/A:1021240730564

[5]Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In Proceedings of the 26th International Conference on World Wide Web (WWW '17). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 173–182. https://doi.org/10.1145/3038912.3052569

[6]https://github.com/malcolmosh/goodbooks-10k-extended

rating, and language. For each user, the dataset provides information about the books they rated and which books they had on their wishlist.

3.2 Exploratory Data Analysis

We performed exploratory data analysis to get a better understanding of the relationships between different features in our dataset. First, we took a look at whether the number of books a user rated influenced the user's average book rating.

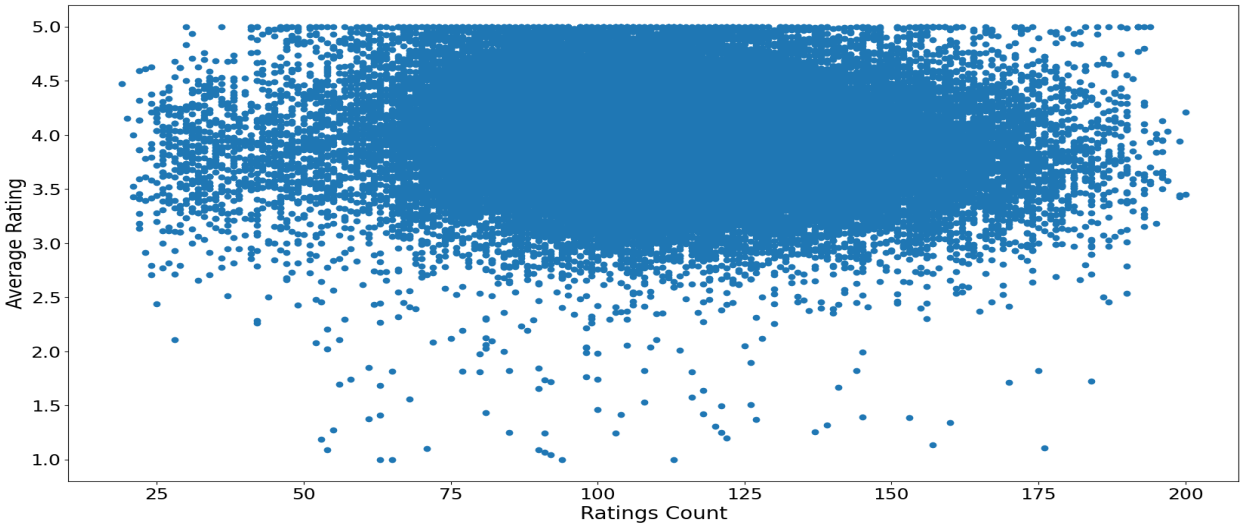


Figure 1: Scatterplot of Average Rating vs. Ratings Count

Figure 1 depicts the relationship between the number of ratings a user has performed (x-axis) and the average rating given by the user (y-axis). Based on the plot, it appears that the number of books a user rated does not have a significant influence on the user's average book rating. Regardless of the number of ratings given, it appears that users generally give an average rating of between 3.0 and 5.0.

We also created a correlation matrix to view how interconnected our variables are. As shown in Figure 2, most of the numerical features in our dataset do not appear to be strongly negatively or positively correlated. Thus, we opted to concentrate on textual features for our content-based filtering model, utilizing book authors, descriptions, genres, and titles to better understand and recommend similar books.

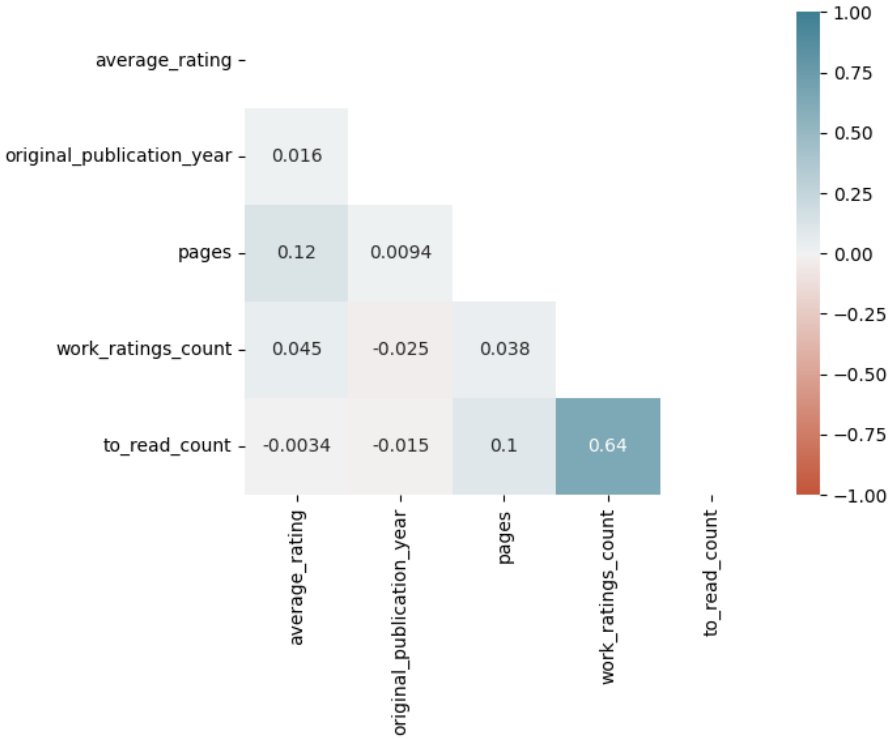


Figure 2: Correlation Matrix of Book Attributes

3.3 Hybrid Model

Our first recommendation model approach was a hybrid recommendation model that integrates content-based filtering and collaborative filtering techniques. This hybrid model leverages the strengths of both techniques to provide more relevant recommendations.

3.3.1 Collaborative Filtering Model

The hybrid model began as a collaborative filtering model that utilizes the K-Nearest Neighbors (KNN) algorithm. This model identifies patterns in user behavior by analyzing the ratings users give to different books. We first constructed a user-item interaction matrix to encapsulate the six million book ratings, where each row corresponds to a user and each column to a book. To reduce bias and account for individual rating habits, we adjusted the ratings by centering them around each user's average rating.

When the model receives a list of books, it computes the average rating profile for those books based on the user-item interaction matrix. Then, the model finds the most similar books to this average profile by using the KNN algorithm with cosine similarity (which measures how similarly users rate different books). We attempted to use Euclidean distances, but it made the model significantly slower (taking 3-4 minutes per recommendation compared to 5-10 seconds with cosine similarity).

Although this approach worked well for popular books, it struggled with less-reviewed books. To address this issue, our hybrid model combines collaborative filtering with content-based filtering, a technique that recommends books based on their actual content (e.g., authors, descriptions, genres).

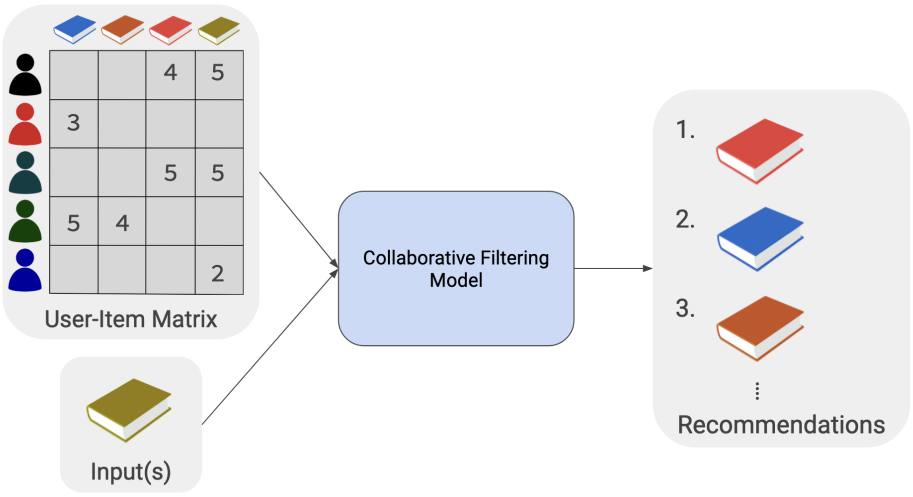


Figure 3: Collaborative Filtering Model

3.3.2 Content-Based Filtering Model

Our content-based filtering model recommends books based on the similarity of authors, descriptions, genres, and titles. We first transformed the textual content of our book dataset into numerical vectors using the Term Frequency-Inverse Document Frequency (TF-IDF) vectorizer. The TF-IDF vectors convert the textual content into numerical values by measuring the importance of each word relative to its frequency in the entire dataset. This highlights significant features of each book's content, enabling us to quantitatively compare the similarities between books.

Next, we used cosine similarity to measure the similarity between books. Our content-based filtering model makes recommendations by computing and comparing all books in the dataset. It first takes in a list of books from the user, calculates the cosine similarity score between these inputted books and all other books in the dataset, and then averages the similarity scores to reflect the overall similarity to all inputted books. Then, the model sorts all other books in the dataset by similarity score and recommends the top N books from the sorted list.

The content-based model provides personalized recommendations by focusing only on the textual features of the books. This approach solves the cold start problem by ensuring that even books with limited ratings can still be recommended based on their content.

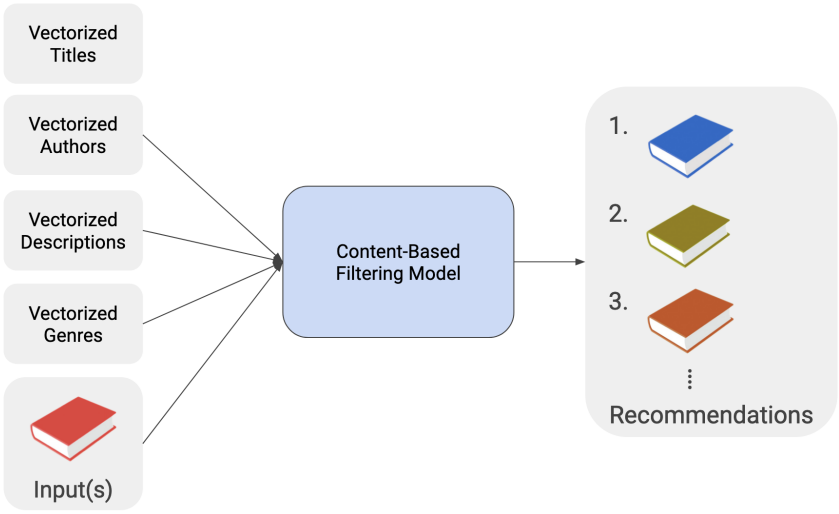


Figure 4: Content-Based Filtering Model

3.3.3 Combining Collaborative Filtering and Content-Based Models

In our hybrid recommendation system, we combined our collaborative and content-based models to improve the relevancy of our recommendations. This approach considers both the user interactions and contextual features of books. To combine the two models, we generate two sets of recommendations: one based on the collaborative filtering model and one based on the content-based model. We sort all of the recommendations based on their similarity score. If a book appears on both models' recommendations, the book's similarity score is summed to reflect higher confidence in the recommendation. We then select the top N books for the final recommendation. By combining both collaborative and content-based methods, our model provides personalized recommendations that account for both user preferences and content features.

3.4 Neural Collaborative Filtering Model

3.4.1 Model Architecture

Our second approach to generating book recommendations was to use Neural Collaborative Filtering (NCF). We defined and trained an NCF model to predict the ratings users would give to books in our dataset. Our NCF model has two embedding layers, one for books and one for users. The main function of the embedding layers is to transform the encoded user and book IDs into dense, low-dimensional vectors. These dense vectors are a computationally efficient way of capturing the essential features that characterize user preferences and book attributes. The dense vectors produced by the embedding layers are concatenated together before being passed to the fully connected layers. This concatenation enables the model to learn complex relationships between user preferences and book characteristics more effectively.

3.4.2 Training

During the forward pass of training, the model takes in user IDs and book IDs as input and outputs predicted ratings. The loss between the predicted ratings and actual ratings is computed using the Mean Squared Error (MSE) function. This loss is then propagated backward, and the model parameters are adjusted using the Adam optimizer. To monitor the training process, we calculated the average training loss, validation loss, RMSE, and MAE at each epoch. To mitigate overfitting, early stopping was implemented to halt training if the validation loss did not improve for five consecutive epochs.

3.4.3 Hyperparameter Tuning

After training, the epoch statistics were plotted and the model's accuracy was qualitatively evaluated using example book inputs. Considerable effort was put into hyperparameter tuning to reduce overfitting and improve model predictions. We experimented with different learning rates, batch sizes, embedding sizes, and model architecture layers. Notably, we discovered that an embedding size of 64 provided optimal results. Additionally, incorporating dropout layers with a rate of 50% significantly reduced the model's tendency to overfit.

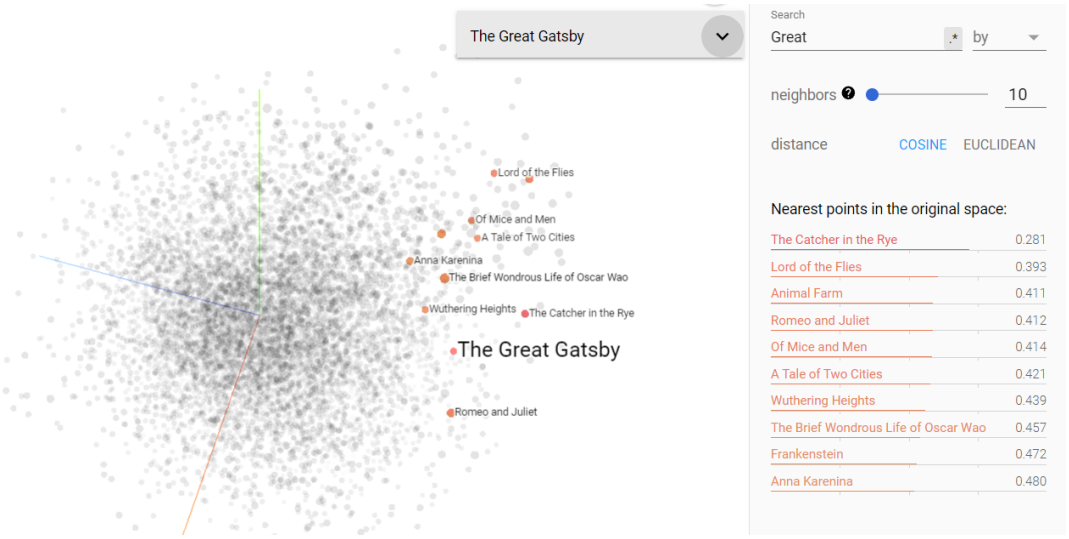


Figure 5: Visualization[7]of Extracted Books Embeddings using TensorBoard Embedding Projector

3.4.4 Generating Recommendations

To generate book recommendations from our trained model, we extracted and normalized the book embeddings. We defined a function that takes in one or more books as input and returns the top "n" books with the highest similarity scores. These scores are determined by computing the dot product between the input book's embedding and all other book embeddings from the model. If multiple books are given as input, we take the average of their embeddings to use in computing the similarity scores.

3.5 Matrix Factorization Model

Our third and final approach was leveraging matrix factorization to predict missing user/book rating pairs. In real-world sets of users and media (including our dataset), it is highly improbable for every user in the set to have rated every book in the set. So, we made use of AI to generate user book ratings for books that they have not read. This gave us a completely filled [users $\times$ books] matrix which is useful because it allowed us to apply simpler algorithms for making recommendations.

3.5.1 Matrix Factorization Overview

We based our model on Simon Funk's well-known implementation of the algorithm created during the 2006 Netflix Prize competition[8]. Funk developed his code in C to work with the Netflix database, so we modified an optimized Python implementation of his code[9] to work with our dataset. The process began with a sparsely filled [users $\times$ books] matrix, only containing entries at indices for which a user has read and

[7]https://projector.tensorflow.org/?config=https://gist.githubusercontent.com/madeleine-clark/e83d015e1cf93ab6f1c3ce641f54600d/raw/91d45705bcb5879bc0ffdcbd5c3ac18ec052e269/book-embedding-visualization.json

[8]https://sifter.org/simon/journal/20061211.html

[9]https://github.com/gbolmier/funk-svd

rated the corresponding book. To predict the missing ratings, the algorithm factored the sparse matrix into two smaller factor matrices. These factors were optimized iteration after iteration as they incrementally converged on the original values in the sparse matrix when multiplied together, closely approximating those original ratings while also filling every empty entry. This solved our issue of missing ratings, but how do we know that these predicted ratings have any meaning?

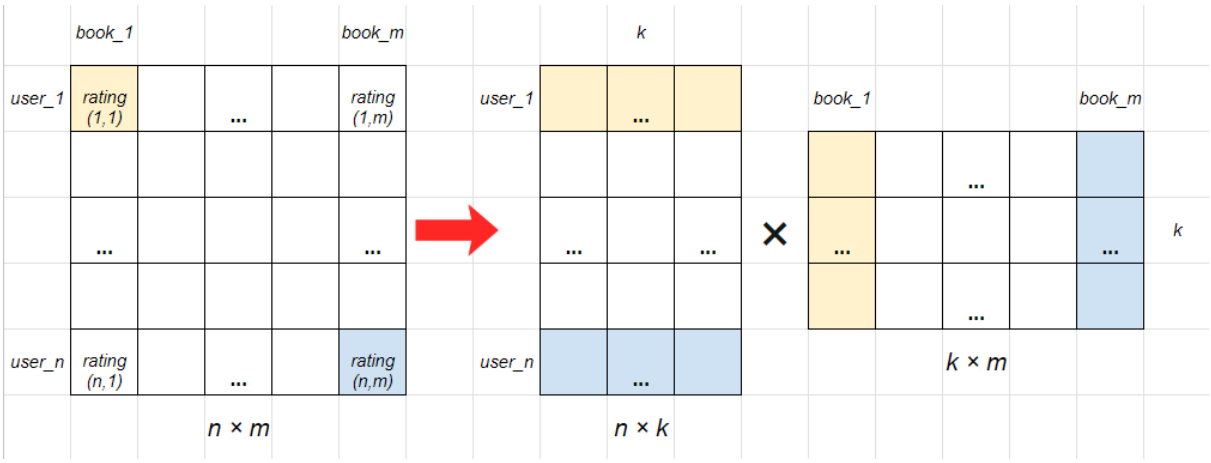


Figure 6: An Example Of Matrix Factorization

Assuming we have n users and m books, the dimensions of the full matrix are $n \times m$, and the factors are $n \times k$ and $k \times m$. This k value represents the number of latent features which are derived by the algorithm. For our implementation, we elected to use 40 latent features. Latent features are implicit characteristics that can describe users and books at an abstract level. The $n \times k$ matrix is the user latent matrix and the $k \times m$ is the book latent matrix, describing n users and m books each with k latent features. User latent features represent their preferences for certain book characteristics and book latent features represent those corresponding characteristics. We cannot interpret these characteristics, but they are derived from the given user/book rating pairs, so extrapolating them to make new book rating predictions for users gives accurate results. If we want to predict a new book rating for a user, we can take the dot product of their row in the user latent matrix and the transpose of the book's column in the book latent matrix. We then add a bias (the average rating of the given user/book pairs) to normalize the predicted rating with the original ratings.

3.5.2 Generating Recommendations

Once the sparse matrix was filled, we took a two-step approach to make recommendations for our new user. The first step was finding a set of users in the dataset similar to our new user. To start, the similar users must have actually rated a number of the new user's inputted books. We hardcoded this number based on what provided the best performance but would have preferred to algorithmically determine it. From this set of similar users, we calculated the mean squared error between the ratings our new user and the similar users gave for the inputted books. We selected the top 50 users with the lowest error values for the next part of the algorithm. The second step was extracting recommendations from these similar users. To do this, we used a combination of collaborative and content-based strategies. We found the average rating of every book in the dataset among the similar users and the number of overlapping book tags the inputted books and every other book. We then normalized these values and added them together, giving more weight to the number of overlapping tags than the average rating to make sure the recommendations were more relevant to the inputted books. Higher scores represent highly rated books which are similar content-wise to our inputted books, so the top 10 are given as recommendations to our new user.

3.6 Natural Language Processing Model

One idea that occurred was, what if we were to use a natural language processing model? Intuitively speaking, it would make sense, since what better way to make a recommendation than to have a model read

the book, define characteristics of the book, and compare it to other books? Although there wasn't enough time to create a convincing model to work, the concepts used for a natural language processing model are still very interesting and something worth looking into.

3.6.1 Natural Language Processing Model Concept

During our research for finding some natural language processing models, a few models stood out. One that would read the descriptions of the book, and another model that was able to classify writing styles. For the model that read the descriptions of the book[10], it used similar algorithms as the content-based filtering model.

In this model, it also used some forms of word embeddings, as well as the TF-IDF method to create numeric vectorization, which later was used for the cosine similarity algorithm, which creates the recommendation. With a little bit of adjustments, the model would then theoretically be able to read entire books and produce very accurate recommendations.

As for the model that was able to classify writing styles, the concept would be relatively similar. Have the model classify some writing style based on the content of the book, then compare different styles from other books to see which books are most similar in terms of their writing style. Then produce a recommendation based on which are the most similar. For the model that was researched[11], the model would find lexical features, vocabulary richness, and readability score from the article. Methods used to find each type of features can be found within the repository. Then, it would use K-means clustering to help group the writing styles, and also differentiate between two different articles writing styles. In theory, we could then recommend the book, or books, that were the most similar within the clustering.

3.6.2 Issues With Natural Language Processing Models

Though in theory, the idea of a natural language processing model would sound great for creating book recommendations, in reality, it also faces issues. One of the big issues would be the sheer quantity of data needed for the model. If we wanted a model to produce convincing recommendations, we would then also need the model to have many many large text files of the contents of the book as data. Then to computate each book's content would be very computationally taxing, and impractical. However, if we were to shrink down the content of the book, we may lose a lot of data required for the method to create a convincing recommendation. Due to time constraints, unable to find a potential middle ground for the amount of content we should feed into the model, and how the amount of computation needed seemed impractical for the objective we desired, the natural language processing model fell short.

3.7 Website Development

Our web application was developed using React for the frontend and Flask for the backend. React was chosen for its efficiency and ease of use, while Flask was selected for its simplicity and compatibility with Python, the language used for our models. This setup allows users to input their favorite books and receive personalized recommendations through a user-friendly interface. The application asks users to input three books that they have previously read and rate highly. These inputs are then fed into a KNN model that implements collaborative filtering. The application then outputs 10 recommendations per book as well as each recommendation's corresponding distance as a measure of cosine similarity. This application also features a "More Information" page that includes important information on the KNN model implemented, key AI methodologies, and challenges faced when designing the model. Due to time constraints, the NCF and hybrid models were unable to be implemented into the application, however we plan to implement these in the near future.

[10] https://github.com/AmoliR/nlp-for-book-recommendation

[11] https://github.com/Hassaan-Elahi/Writing-Styles-Classification-Using-Stylometric-Analysis/blob/master/README.md

4 Results

4.1 Hybrid Model Performance

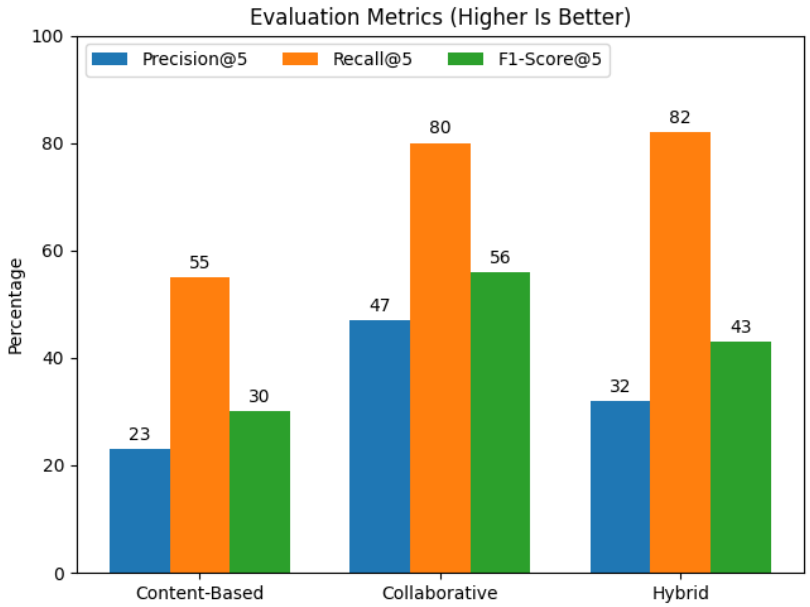


Figure 7: Visualization of Hybrid Model's Performance

We found that the collaborative filtering model produced promising evaluation metrics, performing best with five recommendations: Precision@5 of 0.47, Recall@5 of 0.80, and F1-score@5 of 0.56. Precision@5 indicates that 47% of the top five recommendations were relevant, while Recall@5 shows that 80% of the relevant books were included in the top five. The F1-score@5 balances the precision and recall metrics.

The content-based filtering model had lower performance metrics: Precision@5 of 0.23, Recall@5 of 0.55, and F1-score@5 of 0.30. Despite these results, the content-based model still plays a crucial role in solving the cold start problem.

The hybrid model achieved a similar performance to the collaborative filtering model with Precision@5 of 0.32, Recall@5 of 0.82, and F1-score@5 of 0.43. The lowered precision score is due to the inclusion of the content-based recommendation model, which improves the ability to recommend books with limited user ratings. Combining both models provides more comprehensive and balanced recommendations.

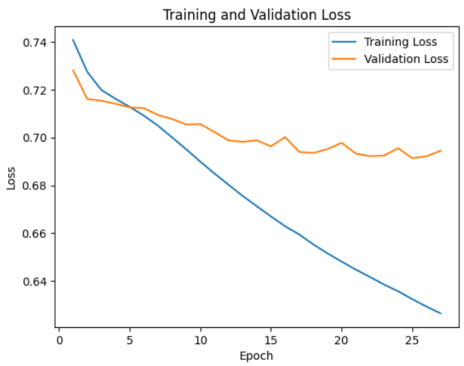
The pattern of having high recall but lower precision indicates that our model is focused more on minimizing false negatives compared to minimizing false positives. This means that our model aims to ensure that as many relevant books as possible are recommended, even if it includes some irrelevant books in the process. The hybrid model's high recall but lower precision may stem from its emphasis on recommending "safe bets" such as books by the same author or within the same series, which are more likely to be relevant but can also limit the diversity of recommendations.

4.2 Neural Collaborative Filtering Model Performance

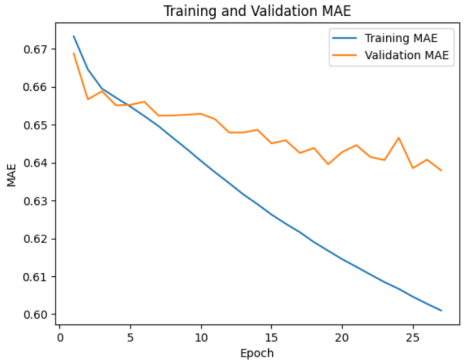
The quantitative metrics used to evaluate the NCF model indicates subpar performance. However, testing the model with example input books produced surprisingly accurate recommendations. Despite extensive hyperparameter tuning, the NCF model remained prone to overfitting. The training loss decreased linearly with each epoch, but the validation loss stopped decreasing significantly around epoch 15 and started to become more volatile. This volatility is more pronounced in the MAE graph, suggesting that the model was making regular, small errors rather than infrequent but large errors.

Additional performance evaluation was computed after training by computing precision, recall, and F1 scores. These scores were computed in a similar manner to the hybrid model, and were recorded as follows:

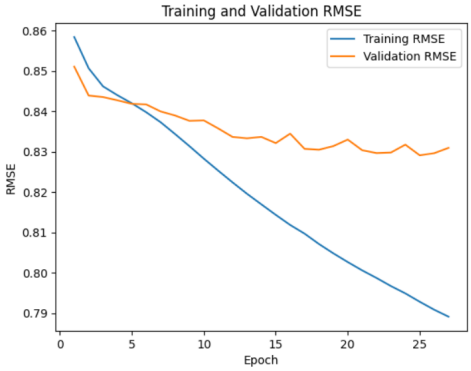
Precision@5 of 0.23, Recall@5 of 0.82, and F1-score@5 of 0.35. This indicates that the NCF model's performance is comparable to the hybrid model's in that it is good at identifying relevant recommendations, but also includes quite a few irrelevant recommendations (i.e. "false positives"). There are many possible explanations for this behavior, but a likely cause is that the model overfitted on popular books, and hence recommended them often even when not inline with a specific user's preferences.



(a) NCF Loss over Epochs



(b) NCF MAE over Epochs



(c) NCF RMSE over Epochs

Figure 8: Visualization of Neural Collaborative Filtering Model's Performance

4.3 Matrix Factorization Model Performance

We split the given ratings into a 70-15-15 training-validation-testing split for evaluating our model. We chose to use two quantitative error metrics to evaluate our model – root mean squared error (RSME) and mean absolute error (MAE). We chose these metrics in particular because they are referenced in Simon Funk's paper, so we wanted to have a basis for comparison in a similar type of system. These metrics compute the relative differences between the given user/book rating pairs and those that were generated by the model for the same user/book pair. We are only able to calculate errors for the given data because there is no quantitative basis from which we can evaluate the predicted ratings.

As a baseline, Funk's model for Netflix achieved a training RSME of 0.9514 and a testing RSME of 0.9525. The winning model in the Netflix Prize competition achieved a training RSME of 0.8554 and a testing RSME of 0.8567. When we chose an arbitrary rating value to fill the matrix, we achieved a training RSME of 0.9910 and a testing RSME of 0.9913. It also achieved a training MAE of 0.7739 and a testing MAE of 0.7743. Finally, our factorized matrix achieved a training RSME of 0.7564 and a testing RSME of 0.8467. It also achieved a training MAE of 0.5876 and a testing MAE of 0.6568.

Our error values were on par with, and even lower than, metrics used in industry-standard implementations (at least from the 2000s). We achieved promising results for factoring the matrix, but extracting recommendations from this matrix was challenging.

5 Discussion

Qualitatively, the hybrid model provides reasonable recommendations, but the recommendations are not very novel and unexpected. Specifically, the hybrid model tends to prioritize recommending books within the same author or series rather than suggesting more diverse works from other authors within the same genre or with similar descriptions. For example, the model's recommendations for *The Hunger Games* included *Catching Fire*, *Mockingjay*, and *The World of the Hunger Games*. For *The Great Gatsby*, recommendations included other works by F. Scott Fitzgerald (e.g., *This Side of Paradise* and *The Short Stories*) and books with thematic similarities like The *Catcher in the Rye*. To improve the recommendations of the hybrid model, our model could give less weight to author similarity and more to genre and description similarities. This would help ensure that users are exposed to a broader range of books.

The recommendations generated by the NCF model are also reasonable but in some ways more insightful. Like the hybrid model, the NCF model recommends books in the same series (e.g. *Twilight* and *The Hunger Games*) and even suggests them in the most logical reading order. However, for stand-alone books not belonging to a niche genre, the NCF model gives non-obvious but thematically similar recommendations. For example, for *The Great Gatsby*, the model recommended *Of Mice and Men*, *Animal Farm*, and *A Tale of Two Cities*. Besides being renowned classic novels, these works share thematic similarities such as the decline of the American Dream, class inequality, and the role of fate. Interestingly, the NCF model did not recommend any other book written by F. Scott Fitzgerald, which makes sense considering that the model was not provided with author information as input.

Ideally, the best recommendations could be obtained by combining the results of the hybrid model with the NCF model. This combination would leverage the strengths of both models, ensuring the relevance and novelty of the recommendations. By integrating these two models, we can create a balanced recommendation system that not only recommends "safe bets" but also introduces users to new and diverse reading options.

The matrix factorization model, while not directly comparable to the other models, is still worth discussing. Our methodology to predict missing ratings through matrix factorization closely replicates former industry-standard models, but we were responsible for creating our own recommendation system from this matrix, which proved to be very difficult. Our model gives a great variety of recommendations, with some being far more relevant than others. For example, when inputting *Fifty Shades of Grey* and *Fifty Shades Darker*, we got recommendations with romance themes also at the forefront like *Me Before You* and *Once and Always*, but also generally popular books like *The Hunger Games* and *Twilight*. Recommending popular books was an issue for all inputs because we had to take average ratings into account given such a simple setting to work with. Disregarding this issue, the system gives a great variety of recommendations, something which certain users may find more useful.

6 Challenges

We faced three primary challenges throughout this project. Firstly, we struggled to find example recommendation models online that do not rely on user data. Most existing recommendation systems are user-centric, making it difficult to find references for a book-based approach. The second primary challenge was addressing the cold start problem for less-reviewed books. Generating recommendations for such books required the integration of content-based methods effectively. The last primary challenge we faced was finding the right balance between diversity and relevance in recommendations. Careful model tuning and evaluation are needed to ensure that the recommendations are both relevant and varied to avoid a monotonous user experience.

7 Conclusion

In this project, we developed three distinct book recommendation systems: a hybrid model combining collaborative and content-based filtering, a Neural Collaborative Filtering (NCF) model, and a matrix factorization model. Each system offers unique strengths in delivering personalized book recommendations. The hybrid model effectively balances user preferences with content similarities, addressing the cold start problem. The NCF model leverages deep learning to capture complex user-item interactions, providing nuanced and insightful recommendations. The matrix factorization model uses a singular simple matrix as its primary medium and provides a variety of recommendations.

Our hybrid model demonstrated a strong ability to provide relevant book recommendations, though it tended to prioritize books by the same author or within the same series. The NCF model offered thematically rich suggestions, especially for standalone books, showcasing the potential for combining different models to enhance recommendation diversity and accuracy. The matrix factorization model proved inferior to the others, often giving irrelevant recommendations and having a hard requirement of multiple input books.

Overall, our project successfully implemented and evaluated three advanced recommendation systems, deepening our understanding of recommendation technologies and their applications in enhancing user experiences in the literary domain. This project highlights the importance of leveraging multiple approaches to create a robust and versatile recommendation system that can cater to diverse user needs.

8 Contributions

8.1 Christina Phan

Christina contributed to the creation of, improvements upon, and evaluations of the hybrid book recommendation model. She also scheduled group meetings and contributed significantly to writing project deliverables throughout the quarter. Additionally, Christina formatted the final report in LaTeX.

8.2 Jack Schonherr

Jack extensively researched matrix factorization models and contributed to the development of the recommendation system in its implementation. He also contributed to writing about the model and developing graphics for it in the final report.

8.3 Lauren Mandell

Lauren contributed to data visualization for exploratory data analysis of our dataset as well as the creation of and improvements upon the hybrid book recommendation model. She also contributed significantly to templating and writing project deliverables throughout the quarter.

8.4 Madeleine Clark

Madeleine researched, designed, developed, evaluated, and spent 40+ hours hyperparameter tuning and training the NCF model. She also made a Notion page to track the team's progress and store the team's resources. She contributed to writing the final report and providing visualizations.

8.5 Sajid Mohammad

Sajid researched Matrix Factorization and found the Python library we used for our dataset. He additionally designed our novel recommendation algorithm and co-developed it with Jack. He created the slides for matrix factorization, including creating graphics from scratch, and helped facilitate group discussion.

8.6 Vedis Wu

Vedis researched, tested, and discovered the faults for a NLP model. He also created the control data, ChatGPT data, and data analysis for the comparison of each model however, to be within the word count, it was removed from the final paper.

8.7 Vibha Chandrasekar

Vibha researched the implementation and found source code for Random Forest models in recommendations systems, however implementing Random Forest models were not necessary for this project. She implemented the frontend and backend for the web application.